SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 2 – Agile Sprint Planning & User Story Workshop

Course Code:	CSA10 / CSA1021	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 2 – **Agile Sprint Planning & User Story Workshop**
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	Ashira Bridget Susan S	**Reg. No.:**	192524316
Date:	14/10/2026	**Duration:**	60 minutes

Code of Conduct

*I, **Ashira Bridget Susan S (!92524316),** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student:



Annexure A – Agile Sprint Planning & User Story Workshop

Instructions to Students:

Each student is assigned an individual software project problem statement. Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

1. **Identify the Project Vision and Stakeholders**
   - Define the project's vision, objectives, and identify all key stakeholders involved.
2. **Create Epics and User Stories**
   - Break down the requirements into Epics and formulate well-structured User Stories using the format: *As a <user>, I want <feature>, so that <benefit>.*
3. **Define Acceptance Criteria**
   - Write clear and measurable acceptance criteria for each selected user story using the Given–When–Then format or equivalent.
4. **Prioritize the Product Backlog**
   - Organize and prioritize the product backlog based on business value, customer needs, and project dependencies.
5. **Plan the Sprint Backlog**
   - Select high-priority user stories for the first sprint and divide them into actionable development tasks.
6. **Estimate Effort Using Story Points**

- Assign story points to each selected user story using an appropriate estimation technique (e.g., Fibonacci sequence or Planning Poker) and justify your estimates.

7. **Present the Sprint Goal and Expected Deliverables**
   - Define a clear sprint goal and describe the expected sprint deliverables, including the features to be completed at the end of the sprint.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Project Vision & Stakeholder Identification (30%)	Project vision is clear, comprehensive, and aligned with the problem statement. All relevant stakeholders and their roles are correctly identified.	Vision is clear with minor omissions. Most stakeholders are correctly identified.	Vision is partially defined. Some stakeholders are missing or incorrectly identified.	Vision is unclear or incomplete. Stakeholders are poorly identified or missing.
Epics & User Stories (25%)	Epics are well-defined and user stories are complete, follow Agile format, and fully capture functional requirements.	Epics and user stories are mostly complete with minor formatting or requirement issues.	User stories are partially complete with limited Agile structure.	User stories are incomplete, unclear, or do not follow Agile practices.
Acceptance Criteria (20%)	Acceptance criteria are specific, measurable, testable, and aligned with all user stories.	Acceptance criteria are mostly clear and testable with minor gaps.	Acceptance criteria are partially defined and lack clarity or completeness.	Acceptance criteria are missing or not aligned with user stories.
Product Backlog Prioritization & Sprint Planning (15%)	Product backlog is logically prioritized, sprint backlog is realistic, and task breakdown is complete.	Backlog prioritization and sprint planning are mostly appropriate with minor issues.	Prioritization or sprint planning is partially complete with limited justification.	Backlog prioritization and sprint planning are poorly organized or incomplete.
Story Point Estimation, Sprint Goal & Presentation (10%)	Story point estimation is well justified, sprint goal is clear, deliverables are realistic, and presentation is professional and well organized.	Estimation and sprint goal are mostly appropriate with minor inconsistencies. Presentation is clear.	Estimation or sprint goal lacks justification. Presentation is adequate but incomplete.	Estimation is unrealistic or missing. Sprint goal, deliverables, and presentation are unclear or incomplete.

Criteria	**Mark**
Project Vision & Stakeholder Identification **(30%)**	/5
Epics & User Stories **(25%)**	/5
Acceptance Criteria **(20%)**	/5
Product Backlog Prioritization & Sprint Planning **(15%)**	/5
Story Point Estimation, Sprint Goal & Presentation **(10%)**	/5
TOTAL	/25

Annexure A – Agile Sprint Planning & User Story Workshop

Project Title

Autonomous Heavy-Duty Truck Platooning Control & Telemetry Synchronization Framework

1. Project Vision and Stakeholders

Project vision. We wrote the vision in the classic "for / who / unlike" style so that the whole team can remember it in one breath:

> For **long-haul fleet operators and their drivers** who want lower fuel costs without giving up safety, **PlatoonSync** is a fault-tolerant control and telemetry framework that lets small convoys of semi-autonomous trucks drive about 10 m apart at 100 km/h, stay in sync within 2 ms, and separate safely the moment the wireless link fails. Unlike ordinary adaptive cruise control, which cannot share braking and throttle state between trucks, PlatoonSync keeps every truck working from the same real-time picture and keeps a tamper-proof record of everything that happens.

Why it is worth building (the evidence). Before planning sprints we checked what real tests say, because that decides what our backlog must protect. The table shows the numbers we are using.

Source	What was found	What it means for us
NREL track tests (2014, two Class 8 trucks)	Lead truck saved up to 5.3% fuel and trailing truck up to 9.7%; team saving 3.7–6.4%, best at 55 mph and a 30 ft (about 9 m) gap.	Our fuel target is realistic at 4–6% for the team, not 30%.
California PATH three-truck platoon	Leader saved about 4% and followers 10–14%.	Followers gain the most, so reporting savings per truck matters (US-13).
NREL / LBNL track campaign (2017)	Across 26 two- and three-truck scenarios, savings of followers dropped at very close gaps.	Gap holding needs a tuned target, not just "as close as possible" (US-04).
EU ENSEMBLE (2018–2022)	Seven truck makers tested multi-brand platoons with time gaps of 0.3–1.2 s; the gap grows when a cut-in is detected.	Cut-in handling and shared message formats belong in the backlog (US-10, US-11).
Canadian winter commercial trial (SAE)	Cut-ins rose from 1.6 to 5 per hour when the gap grew from 3 to 5 s; no substantial fuel saving was seen overall.	Real roads are harder than test tracks, so we plan for dropouts and cut-ins early.

Where the world is now. Interest is moving from demos to real use: the agenda of the 2026 North American Autonomous Truck Conference talks about a shift from pilots to commercial leader–follower platooning, and a Minnesota bill introduced in March 2026 proposes a platooning pilot of up to three vehicles until 2030. So the timing for a project like this is good.

Objectives (made measurable).

- Keep the control loop of every truck in sync within a 2 ms timing budget.
- Prove string stability: a braking disturbance must not grow from one truck to the next.
- Complete form, split, lane-change and merge maneuvers only after every truck acknowledges.
- Declare link loss after 3 missed heartbeats (about 6 ms) and separate safely.
- Keep a hash-chained black-box log with 100% tamper detection in tests.
- Reach zero rear-end collisions in 1,000 simulated emergency-brake runs and a 4–6% team fuel saving on the test track.

Stakeholders. In Scrum terms, the fleet operations manager acts as Product Owner because they speak for the business value.

Stakeholder	What they want	Role in the project
Fleet operations manager	Lower fuel cost, on-time delivery	Product Owner
Lead and follower drivers	Safe, easy-to-trust system and clear alerts	Users, give feedback in reviews
Development team (4 members)	Clear stories and stable requirements	Build and test the system
Scrum Master	A team that is not blocked	Runs the sprint events
Test and safety engineers	Testable criteria, simulation access	Check every Definition of Done
Highway authority and insurers	Proof of safety, trustworthy logs	Reviewers and regulators
Network providers	Clear latency and coverage needs	Supply wireless links

2. Create Epics and User Stories

The requirements break down into six epics. Under each epic we wrote small user stories in the standard format, each one small enough to finish in a single sprint. Figure 1 shows the whole product on one page as a story map, with the colours telling us in which sprint each story is planned.

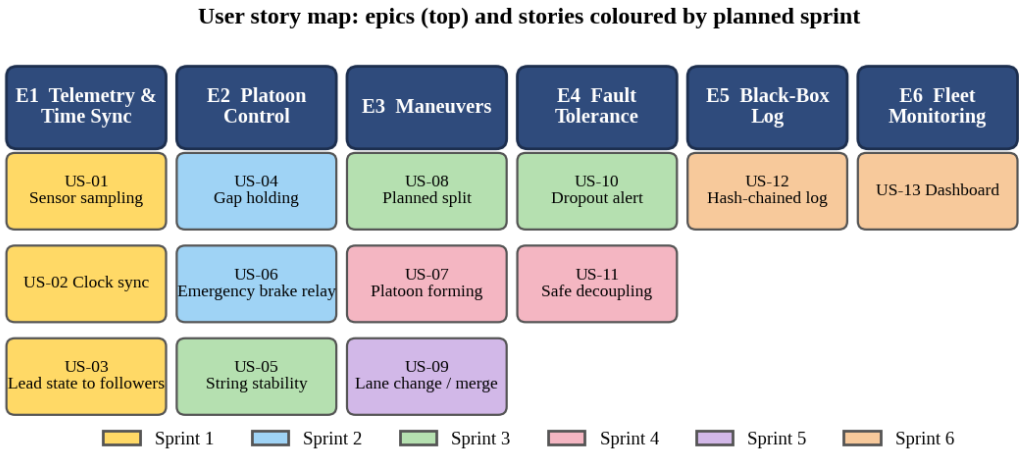


Figure 1: User story map of PlatoonSync

ID	User story
Epic 1: Telemetry and Time Synchronization	
US-01	As a control engineer, I want radar distance, wheel speed, brake pressure and engine torque sampled and timestamped together, so that the controller works from one consistent picture of the truck.
US-02	As a control engineer, I want the clocks of all trucks synchronized to a common time source, so that telemetry from different trucks can be compared inside the 2 ms budget.
US-03	As a lead-truck driver, I want my speed, throttle and brake state sent to every follower immediately, so that they react to me before their radar sees the gap change.
Epic 2: Platoon Control	
US-04	As a follower-truck driver, I want my truck to hold the set gap automatically, so that I don't have to brake and accelerate by hand at a 10 m gap.
US-05	As a fleet operator, I want a braking wave to shrink, not grow, as it moves down the line, so that the last truck never brakes harder than the first.
US-06	As a lead-truck driver, I want an emergency brake to reach all followers at once, so that nobody rear-ends the truck in front.
Epic 3: Platoon Maneuvers	
US-07	As a lead-truck driver, I want to form a platoon with a nearby truck, so that we can start saving fuel together.

ID	User story
US-08	As a follower-truck driver, I want a planned split at my exit, so that I can leave the platoon safely.
US-09	As a lead-truck driver, I want lane changes and merges coordinated across all trucks, so that the platoon doesn't break up in highway traffic.
Epic 4: Fault Tolerance	
US-10	As a follower-truck driver, I want an instant alert when the wireless link drops, so that I am ready to take over.
US-11	As a fleet operator, I want a dropout to trigger a safe, automatic separation, so that safety never depends on a human reacting in milliseconds.
Epic 5: Black-Box Logging	
US-12	As an accident investigator, I want a tamper-proof log kept on every truck, so that I can rebuild exactly what happened.
Epic 6: Fleet Monitoring	
US-13	As a fleet operations manager, I want a dashboard of gap, link health and fuel use, so that I can track how well each platoon performs.

3. Define Acceptance Criteria

Acceptance criteria use the Given–When–Then format and carry numbers, so a tester can say “pass” or “fail” without arguing. Many numbers come from the gap itself: at 100 km/h a truck covers about 27.8 m every second, so our 10 m target is only a 0.36 s time gap (Figure 2), far below the 1.5 s of normal cruise control and below the 0.3–1.2 s range tested in ENSEMBLE.

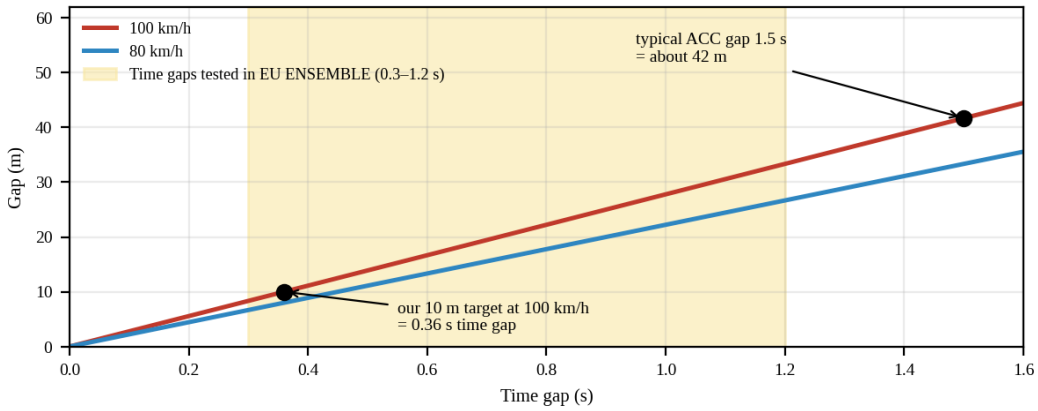


Figure 2: What a time gap means in metres (gap = speed × time gap)

ID	Given	When	Then
US-01	all four sensors are powered and publishing	a sampling cycle completes	one record with radar distance, wheel speed, brake pressure and torque is timestamped and reaches the controller within 2 ms.
US-02	trucks are linked to the common time source	clocks are compared during 10 minutes of driving	the offset between any two trucks stays below 0.5 ms, and larger drift raises an alert.
US-03	the lead truck is in platoon mode	it brakes, throttles or changes speed	every follower receives the state with a sequence number and timestamp, and any message later than 2 ms is flagged and logged.
US-04	a follower with a 10 m target gap at 100 km/h	the lead holds a constant speed on flat ground in simulation	the gap stays within ±1 m of the target.
US-05	a simulated platoon of four trucks	the lead applies a sudden braking step	the peak spacing error never grows from one truck to the next.
US-06	a platoon cruising at 100 km/h	the lead makes an emergency stop	all followers start braking before their radar sees the gap shrink by 1 m, with no collision in 1,000 simulated runs.
US-07	two trucks within wireless range	the lead driver requests a join and the other driver accepts	both trucks authenticate, the follower closes the gap gradually, and the platoon counts as formed only after all acknowledgements.

ID	Given	When	Then
US-08	an active platoon	a follower's driver requests a planned split	that truck opens its gap to at least a 1.5 s time gap before control goes back to the driver.
US-09	a platoon that wants to change lane	the lead sends the maneuver request	all trucks acknowledge, the gap opens, and if one acknowledgement is missing within the timeout the maneuver is cancelled.
US-10	an active wireless link	3 heartbeats in a row are missed	link loss is declared at about 6 ms and the driver alert shows at once.
US-11	link loss has been declared	the follower's controller reacts	it falls back to radar-only cruise control and widens its gap without braking harder than 2 m/s^2.
US-12	a safety event has happened	it is written to the log	the entry holds the hash of the previous entry and a signature, is copied to all trucks, and editing an old entry makes verification fail.
US-13	trucks are reporting to the dashboard	the manager opens it	gap, mode, link health and fuel per 100 km refresh at least every second, and "no data" shows when offline.

4. Prioritize the Product Backlog

We ordered the backlog using MoSCoW (Must, Should, Could, Won't) and three questions: how much business value does it bring, how risky is it for safety, and what does it depend on? Safety and dependencies decided most of the order.

Rank	User story	Priority	Reason
1	US-01 Sensor sampling	Must	Every other feature depends on clean, timestamped data.
2	US-02 Clock sync	Must	Without a shared clock the 2 ms goal means nothing.
3	US-03 Lead state sharing	Must	Gives followers early warning that radar alone cannot.
4	US-04 Gap holding	Must	The core fuel-saving behaviour.
5	US-06 Emergency brake relay	Must	Biggest safety risk at a 0.36 s gap, so it is tested early.
6	US-05 String stability	Must	Stops the "slinky" effect; needs US-04 first.
7	US-10 Dropout alert	Must	Safety net for every later feature.
8	US-08 Planned split	Should	Needed for exits; simple once control works.
9	US-11 Safe decoupling	Must	Completes the fault-tolerance objective.
10	US-07 Platoon forming	Should	Needs gap holding and authenticated messages.
11	US-09 Lane change / merge	Should	Most complex; builds on forming, splitting and dropout handling.
12	US-12 Hash-chained log	Must	Needs events from earlier features; its format is fixed in Sprint 1.
13	US-13 Dashboard	Could	Helpful for operators, but nothing else depends on it.

Extras such as weather-aware gap tuning and platoons of more than five trucks are marked Won't (for now) and left for later releases.

5. Plan the Sprint Backlog

Sprint 1 setup. Sprint length is 2 weeks (10 working days) with a team of 4. We assume about 5 focused hours per person per day, minus meetings, which gives a capacity of roughly 180 hours. **Sprint 1 goal:** get trustworthy, timestamped telemetry flowing from the lead truck to the followers in a simulator. **Selected stories:** US-01, US-02 and US-03.

Task	Description	Story	Hours
T1	Set up repository, automated build and a simple truck simulator	All	12
T2	Define the telemetry message format (fields, units, timestamp, sequence number, event codes)	US-01, US-03	8
T3	Write sensor adapters for radar, wheel speed, brake pressure and torque	US-01	24
T4	Add timestamping and a shared sample buffer running at a fixed 500 Hz	US-01	16
T5	Build the clock-sync service and a drift monitor	US-02	24
T6	Build the lead-to-follower message layer with sequence numbers	US-03	24
T7	Create a latency and jitter measuring tool	US-02, US-03	12
T8	Write unit and integration tests, at least 20 cases	All	20
T9	Review code and fix defects	All	12
T10	Prepare the demo, sprint review and retrospective	All	8
	Total		160

The 160 planned hours leave about 20 hours of buffer, because the first sprint of a new team always has surprises.

6. Estimate Effort Using Story Points

We used the Fibonacci scale (1, 2, 3, 5, 8, 13) with Planning Poker. Everyone shows a card at the same moment so that nobody copies the senior voice. US-01 is our reference story at 5 points, and every other story is compared to it: smaller, similar or bigger.

User story	Points	Justification
US-01 Sensor sampling	5	Four sensor types, timestamping and buffering (the reference story).
US-02 Clock sync	5	Drift handling and testing are tricky but well understood.
US-03 Lead state sharing	3	Reuses the message format from US-01; mostly sending and receiving.
US-04 Gap holding	8	New control law, tuning and many simulation runs.
US-05 String stability	8	Needs analysis and multi-truck tests, with real risk of rework.
US-06 Emergency brake relay	5	Simple message, but safety testing is heavy.
US-07 Platoon forming	5	Handshake, authentication and gradual gap closing.
US-08 Planned split	3	Mostly a variation of gap control.
US-09 Lane change / merge	13	Needs a multi-truck protocol, timeouts and cancellation.
US-10 Dropout alert	3	Heartbeat counter and alert.
US-11 Safe decoupling	8	Fallback logic, gap widening and comfort limits.
US-12 Hash-chained log	8	Hashing, signing and copying to all trucks.
US-13 Dashboard	5	Data feed and screens, with no safety logic.

Totals and velocity. Sprint 1 holds 13 points (US-01: 5, US-02: 5, US-03: 3) and the whole backlog is 79 points. We assume a velocity of 13 points per sprint, which is deliberately cautious for a new team, and we will correct it after Sprint 1. At that rate the release needs six sprints, about 12 weeks (Figure 3). US-09 is 13 points, so it is too big for comfort; before Sprint 5 we will split it into “request and acknowledge” and “open gap and move”.

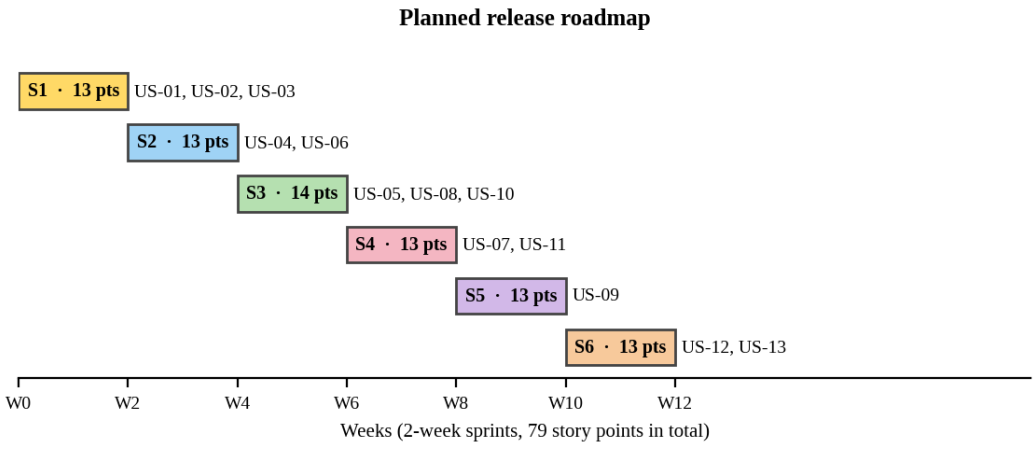


Figure 3: Planned release roadmap based on a velocity of 13 points

7. Present the Sprint Goal and Expected Deliverables

> **Sprint 1 goal:** By the end of two weeks, a simulated lead truck sends timestamped telemetry (radar distance, wheel speed, brake pressure and torque) to follower trucks over a synchronized link, and every message's delay is measured and logged.

Expected deliverables.

- Sensor modules for radar, wheel speed, brake pressure and torque.
- Timestamped sampling at a fixed 500 Hz.
- A clock-sync service with a drift alert.
- A lead-to-follower message layer with sequence numbers.
- A latency and jitter report from at least 20 test cases.
- A working simulator demo shown at the sprint review.

Definition of Done. A story is done when it meets its acceptance criteria, all planned tests pass, there is no open critical defect, the code is reviewed and merged, and it has been demonstrated to the Product Owner.

Conclusion. The workshop turns a big, safety-critical idea into thirteen small stories, six epics and a clear 12-week path. Sprint 1 starts with data and timing because every other feature, from string stability to the black box, depends on them. Real tests tell us to expect 4–6% team fuel savings and to treat dropouts and cut-ins seriously, so the plan builds safety in early.